

Sqrt Tree

Sqrt Tree

引入

给你一个长度为 n 的序列，再给你一个满足结合律的运算（比如 均满足结合律），然后对于每一次区间询问，我们需要计算。

Sqrt Tree 可以在 $\sqrt{n}$ 的时间内预处理，并在 $\sqrt{n}$ 的时间内回答询问。

解释

序列分块

首先我们把整个序列分成 $\sqrt{n}$ 个块，每一块的大小为 $\sqrt{n}$ 。对于每个块，我们计算：

1. 块内的前缀区间询问
2. 块内的后缀区间询问
3. 维护一个额外的数组 ans 表示第 i 个块到第 j 个块的区间答案。

举个例子，假设 $+$ 代表加法运算，序列为 $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$ 。

首先我们将序列分成三块，变成了 $[1, 3], [4, 6], [7, 9]$ 。

那么每一块的前缀区间答案和后缀区间答案分别为

数组为：

（对于 n 的不合法的情况我们假设答案为 0）

显然我们可以在 $\sqrt{n}$ 的时间内预处理这些值，空间复杂度同样是 $\sqrt{n}$ 。处理好之后，我们可以利用它们可以在 $\sqrt{n}$ 的时间内回答一些跨块的询问。但对于那些整个区间都在一个块内的询问我们仍不能处理，因此我们还需要处理一些东西。

构建一棵树

容易想到我们在每个块内递归地构造上述结构以支持块内的查询。对于大小为 $\sqrt{n}$ 的块我们可以 $\sqrt{n}$ 地回答询问。这样我们就建出了一棵树，每一个结点代表序列的一个区间。叶子结点的区间长度为 $\sqrt{n}$ 或 $\sqrt{n}-1$ 。一个大小为 $\sqrt{n}$ 的结点有 $\sqrt{n}$ 个子节点，于是整棵树的高度是 $\sqrt{n}$ 的，每一层的区间总长是 n 的，因此我们构建这棵树的复杂度是 $\sqrt{n}$ 的。

► 树高度的证明

现在我们可以 $\sqrt{n}$ 的时间内回答询问。对于询问 $[l, r]$ ，我们只需要快速找到一个区间长度最小的结点 $node$ 使得 $node$ 能包含 $[l, r]$ ，这样在 $node$ 的分块区间中一定是跨块的，就可以 $\sqrt{n}$ 地计算答案了。查询一次的总体复杂度是 $\sqrt{n}$ ，因为树高是 $\sqrt{n}$ 的。不过我们仍可以优化这个过程。

优化询问复杂度

容易想到二分高度，然后可以 $\sqrt{n}$ 判断是否合法。这样复杂度就变成了 $\sqrt{n}$ 。不过我们仍可以进一步加速这一过程。

我们假设

1. 每一块的大小都是 2^k 的整数幂次；
2. 每一层上的块大小是相同的。

为此我们需要在序列的末位补充一些元素，使得它的长度变成 2^k 的整数次幂。尽管有些块可能会变成原来的两倍大小，但这样仍是 $O(1)$ 的，于是预处理分块的复杂度仍是 $O(1)$ 的。

现在我们可以轻松地确定一个询问区间是否被整个地包含在一个块中。对于区间 $[l, r]$ （以 0 为起点），我们把端点写为二进制形式。举一个例子，对于 $[1, 5]$ ，二进制表示为

我们知道每一层的区间长度是相同的，而分块的大小也是相同的（在上述示例中）。这些块完全覆盖了整个序列，因此第一块代表的元素为 $[0, 1]$ （二进制表示为 $[0, 1]$ ），第二个块代表的元素区间为 $[2, 3]$ （二进制表示为 $[10, 11]$ ），以此类推。我们发现这些在同一个块内的元素的位置在二进制上只有后 k 位不同（上述示例中）。而示例的 $[1, 5]$ 也只有后 k 位不同，因此他们在同一个块中。

因此我们需要检查区间两个端点是否只有后 k 位不同，即 $(l \oplus r) \ll 2^k$ 。因此我们可以快速找到答案区间所在的层：

1. 对于每个 k ，我们找到 2^k 最高位上的；
2. 现在对于一个询问 $[l, r]$ ，我们计算 $(l \oplus r)$ 的最高位，这样就可以快速确定答案区间所在的层。

这样我们就可以在 $O(\log n)$ 的时间内回答询问啦。

更新元素的过程

我们可以在 Sqrt Tree 上更新元素，单点修改和区间修改都是支持的。

单点修改

考虑一次单点赋值操作，我们希望高效更新这个操作的信息。

朴素实现

首先我们来看看在做了一次单点修改后 Sqrt Tree 会变成什么样子。

考虑一个长度为 n 的结点以及对应的序列： $1, 2, \dots, n$ 。容易发现在 l 和 r 中都只有 $O(\sqrt{n})$ 个元素改变。而在 l, r 中则有 $O(\sqrt{n})$ 个元素被改变。因此有 $O(\sqrt{n})$ 个元素在树上被更新。因此在 Sqrt Tree 上单点修改的复杂度是 $O(\sqrt{n})$ 。

使用 Sqrt Tree 替代 B 数组

注意到单点更新的瓶颈在于更新根结点的。因此我们尝试用另一个 Sqrt Tree 代替根结点的，称其为 $Root$ 。它的作用和原来的二维数组一样，维护整段询问的答案。其他非根结点仍然使用 l, r 维护。注意，如果一个 Sqrt Tree 根结点有 l, r 结构，称其 Sqrt Tree 是 **含有索引** 的；如果一个 Sqrt Tree 的根结点有 l, r 结构，称其是 **没有索引** 的。而这棵树本身是没有索引的。

因此我们可以这样更新 $Root$ 树：

1. 在 $O(\sqrt{n})$ 的时间内更新 l 和 r 。
2. 更新 $Root$ ，它的长度是 n 的，但我们只需要更新其中的一个元素（这个元素代表了被改变的块），这一步的时间复杂度是 $O(\sqrt{n})$ 的（使用朴素实现的算法）。
3. 进入产生变化的子节点并使用朴素实现的算法在 $O(\sqrt{n})$ 的时间内更新信息。

注意，查询的复杂度仍是 $O(\sqrt{n})$ 的，因为我们最多使用 $Root$ 树一次。于是单点修改的复杂度就是 $O(\sqrt{n})$ 的。

更新一个区间

Sqrt Tree 也支持区间覆盖操作，即把区间的数全部变成 x 。对此我们有两种实现方式，其中一种会花费 $O(\sqrt{n})$ 的复杂度更新信息， $O(1)$ 的时间查询；另一种则是 $O(1)$ 更新信息，但查询的时间会增加到 $O(\sqrt{n})$ 。

我们可以像线段树一样在 Sqrt Tree 上打懒标记。但是在 Sqrt Tree 上有一点不同。因为下传一个结点的懒标记，复杂度可能达到 $O(\sqrt{n})$ ，因此我们不是在询问的时候下传标记，而是看父节点是否有标记，如果有标记就把它下传。

第一种实现

在第一种实现中，我们只会给第 $\sqrt{n}$ 层的结点（结点区间长度为 $\sqrt{n}$ ）打懒标记，在下传标记的时候直接更新整个子树，复杂度为 $O(\sqrt{n})$ 。操作过程如下：

1. 考虑第 $\sqrt{n}$ 层上的结点，对于那些被修改区间完全包含的结点，给他们打一个懒标记；
2. 有两个块只有部分区间被覆盖，我们直接在 $O(\sqrt{n})$ 的时间内 **重建** 这两个块。如果它本身带有之前修改的懒标记，就在重建的时候顺便下传标记；
3. 更新根结点的 sum 和 max ，时间复杂度 $O(\sqrt{n})$ ；
4. 重建 $\sqrt{n}$ 棵树，时间复杂度 $O(\sqrt{n})$ 。

现在我们可以高效完成区间修改了。那么如何利用懒标记回答询问？操作如下：

1. 如果我们的询问被包含在一个有懒标记的块内，可以利用懒标记计算答案；
2. 如果我们的询问包含多个块，那么我们只需要关心最左边和最右边不完整块的答案。中间的块的答案可以在 $\sqrt{n}$ 棵树中查询（因为 $\sqrt{n}$ 棵树在每次修改完后会重建），复杂度是 $O(\sqrt{n})$ 。

因此询问的复杂度仍为 $O(\sqrt{n})$ 。

第二种实现

在这种实现中，每一个结点都可以被打上懒标记。因此在处理一个询问的时候，我们需要考虑祖先中的懒标记，那么查询的复杂度将变成 $O(\sqrt{n})$ 。不过更新信息的复杂度就会变得更快。操作如下：

1. 被修改区间完全包含的块，我们把懒标记添加到这些块上，复杂度 $O(\sqrt{n})$ ；
2. 被修改区间部分覆盖的块，更新 sum 和 max ，复杂度 $O(\sqrt{n})$ （因为只有两个被修改的块）；
3. 更新 $\sqrt{n}$ 棵树，复杂度 $O(\sqrt{n})$ （使用同样的更新算法）；
4. 对于没有索引的子树更新他们的；
5. 递归地更新两个没有被完全覆盖的区间。

时间复杂度是 $O(\sqrt{n})$ 。

实现

下面的实现在 $O(\sqrt{n})$ 的时间内建树，在 $O(1)$ 的时间内回答询问，在 $O(\sqrt{n})$ 的时间内单点修改。

```
1 SqrtTreeItem op(const SqrtTreeItem &a, const SqrtTreeItem &b);
2
3 int log2Up(int n) {
4     int res = 0;
5     while ((1 << res) < n) {
6         res++;
7     }
8     return res;
9 }
10
11 class SqrtTree {
```

```

12 private:
13     int n, lg, indexSz;
14     vector<SqrtTreeItem> v;
15     vector<int> clz, layers, onLayer;
16     vector<vector<SqrtTreeItem>> pref, suf, between;
17
18     void buildBlock(int layer, int l, int r) {
19         pref[layer][l] = v[l];
20         for (int i = l + 1; i < r; i++) {
21             pref[layer][i] = op(pref[layer][i - 1], v[i]);
22         }
23         suf[layer][r - 1] = v[r - 1];
24         for (int i = r - 2; i >= l; i--) {
25             suf[layer][i] = op(v[i], suf[layer][i + 1]);
26         }
27     }
28
29     void buildBetween(int layer, int lBound, int rBound, int betweenOffs) {
30         int bSzLog = (layers[layer] + 1) >> 1;
31         int bCntLog = layers[layer] >> 1;
32         int bSz = 1 << bSzLog;
33         int bCnt = (rBound - lBound + bSz - 1) >> bSzLog;
34         for (int i = 0; i < bCnt; i++) {
35             SqrtTreeItem ans;
36             for (int j = i; j < bCnt; j++) {
37                 SqrtTreeItem add = suf[layer][lBound + (j << bSzLog)];
38                 ans = (i == j) ? add : op(ans, add);
39                 between[layer - 1][betweenOffs + lBound + (i << bCntLog) + j] = ans;
40             }
41         }
42     }
43
44     void buildBetweenZero() {
45         int bSzLog = (lg + 1) >> 1;
46         for (int i = 0; i < indexSz; i++) {
47             v[n + i] = suf[0][i << bSzLog];
48         }
49         build(1, n, n + indexSz, (1 << lg) - n);
50     }
51
52     void updateBetweenZero(int bid) {
53         int bSzLog = (lg + 1) >> 1;
54         v[n + bid] = suf[0][bid << bSzLog];
55         update(1, n, n + indexSz, (1 << lg) - n, n + bid);
56     }
57
58     void build(int layer, int lBound, int rBound, int betweenOffs) {
59         if (layer >= (int)layers.size()) {
60             return;
61         }
62         int bSz = 1 << ((layers[layer] + 1) >> 1);
63         for (int l = lBound; l < rBound; l += bSz) {
64             int r = min(l + bSz, rBound);
65             buildBlock(layer, l, r);
66             build(layer + 1, l, r, betweenOffs);
67         }
68         if (layer == 0) {
69             buildBetweenZero();
70         } else {
71             buildBetween(layer, lBound, rBound, betweenOffs);
72         }
73     }
74
75     void update(int layer, int lBound, int rBound, int betweenOffs, int x) {
76         if (layer >= (int)layers.size()) {

```

```

77     return;
78 }
79 int bSzLog = (layers[layer] + 1) >> 1;
80 int bSz = 1 << bSzLog;
81 int blockIdx = (x - lBound) >> bSzLog;
82 int l = lBound + (blockIdx << bSzLog);
83 int r = min(l + bSz, rBound);
84 buildBlock(layer, l, r);
85 if (layer == 0) {
86     updateBetweenZero(blockIdx);
87 } else {
88     buildBetween(layer, lBound, rBound, betweenOffs);
89 }
90 update(layer + 1, l, r, betweenOffs, x);
91 }
92
93 SqrtTreeItem query(int l, int r, int betweenOffs, int base) {
94     if (l == r) {
95         return v[l];
96     }
97     if (l + 1 == r) {
98         return op(v[l], v[r]);
99     }
100     int layer = onLayer[clz[(l - base) ^ (r - base)]];
101     int bSzLog = (layers[layer] + 1) >> 1;
102     int bCntLog = layers[layer] >> 1;
103     int lBound = ((l - base) >> layers[layer]) << layers[layer] + base;
104     int lBlock = ((l - lBound) >> bSzLog) + 1;
105     int rBlock = ((r - lBound) >> bSzLog) - 1;
106     SqrtTreeItem ans = suf[layer][l];
107     if (lBlock <= rBlock) {
108         SqrtTreeItem add =
109             (layer == 0) ? (query(n + lBlock, n + rBlock, (1 << lg) - n, n))
110                 : (between[layer - 1][betweenOffs + lBound +
111                     (lBlock << bCntLog) + rBlock]);
112         ans = op(ans, add);
113     }
114     ans = op(ans, pref[layer][r]);
115     return ans;
116 }
117
118 public:
119 SqrtTreeItem query(int l, int r) { return query(l, r, 0, 0); }
120
121 void update(int x, const SqrtTreeItem &item) {
122     v[x] = item;
123     update(0, 0, n, 0, x);
124 }
125
126 SqrtTree(const vector<SqrtTreeItem> &a)
127     : n((int)a.size()), lg(log2Up(n)), v(a), clz(1 << lg), onLayer(lg + 1) {
128     clz[0] = 0;
129     for (int i = 1; i < (int)clz.size(); i++) {
130         clz[i] = clz[i >> 1] + 1;
131     }
132     int tlg = lg;
133     while (tlg > 1) {
134         onLayer[tlg] = (int)layers.size();
135         layers.push_back(tlg);
136         tlg = (tlg + 1) >> 1;
137     }
138     for (int i = lg - 1; i >= 0; i--) {
139         onLayer[i] = max(onLayer[i], onLayer[i + 1]);
140     }
141     int betweenLayers = max(0, (int)layers.size() - 1);

```

```
142     int bSzLog = (lg + 1) >> 1;
143     int bSz = 1 << bSzLog;
144     indexSz = (n + bSz - 1) >> bSzLog;
145     v.resize(n + indexSz);
146     pref.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
147     suf.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
148     between.assign(betweenLayers, vector<SqrtTreeItem>((1 << lg) + bSz));
149     build(0, 0, n, 0);
150 }
151 };
```

习题

[CodeChef - SEGPROD](#)

本页面主要译自 [Sqrt Tree - Algorithms for Competitive Programming](#), 版权协议为 **CC-BY-SA 4.0**。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Chrogeek](#), [ChungZH](#), [Enter-tainer](#), [iamtwz](#), [mgt](#), [ouuan](#), [PerfectPan](#), [qwqAutomaton](#), [sshwy](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用